FaceLit

Artículo del proyecto

Sistema de gestión de asistencia académica con reconocimiento facial

Maria José Rodriguez Oyola

Valery Sinaí Trujillo Quintero

Mariana Valenzuela Penagos

SENA

Análisis y Desarrollo de Software
Instructor: José de Jesús Motta
Octubre de 2026

Índice

Este artículo presenta el problema, la metodología, la solución técnica y los resultados documentados de FaceLit. Incluye el papel de Face Mesh y distingue sus funciones de la comparación de identidad realizada con InsightFace.

Contenido	Página
Título autores resumen y palabras clave	3
1. Introducción y problema abordado	4
2. Metodología de desarrollo	5
3. Desarrollo de la solución	6
4. Face Mesh y reconocimiento facial	7
5. Resultados y discusión	8
6. Conclusiones	8
Referencias bibliográficas	9

FaceLit
Sistema de gestión de asistencia académica con reconocimiento facial

Maria José Rodriguez Oyola · Valery Sinaí Trujillo Quintero
Mariana Valenzuela Penagos

SENA · Análisis y Desarrollo de Software
Instructor: José de Jesús Motta
Artículo técnico del proyecto · Octubre de 2026

Resumen

FaceLit aborda la organización del registro de asistencia académica mediante una aplicación con tres roles: coordinador, instructor y aprendiz. La solución integra un frontend desarrollado con Expo y React Native, un backend Java basado en Spring Boot, una base de datos PostgreSQL y un microservicio Python para el procesamiento facial. El backend se organiza como monolito modular con capas internas y la persistencia incorpora migraciones Liquibase y vectores faciales con pgvector. Según la información actualizada del equipo, Face Mesh se utiliza como apoyo al análisis de la malla facial durante la captura; se distingue de la generación y comparación de embeddings realizada con InsightFace. La metodología articula requisitos, modelado, desarrollo por funcionalidades e integración de componentes. Los resultados descritos corresponden a artefactos y capacidades documentadas, junto con correcciones reportadas por el equipo; no se presentan mediciones experimentales de precisión o rendimiento. Se concluye que la solución ofrece una base técnica para el seguimiento de asistencia, cuya eficacia requiere validación integral en condiciones reales.

Palabras clave

Asistencia académica; reconocimiento facial; Face Mesh; monolito modular; microservicio; PostgreSQL; aplicación multiplataforma.

1. Introducción y problema abordado

El seguimiento de la asistencia requiere información consistente sobre los aprendices, sus fichas y las sesiones de formación. Un procedimiento manual o disperso puede producir omisiones, registros duplicados y dificultades para consultar el historial. FaceLit se desarrolla para reunir la administración académica y el registro de asistencia en una solución accesible desde una interfaz web y una aplicación Android.

El reconocimiento facial es un componente del proceso, pero no reemplaza la gestión académica. Una captura necesita un usuario registrado, una asociación con la ficha y un contexto de sesión. Además, el acceso debe diferenciar quién administra la información, quién opera una sesión y quién consulta datos propios.

Problema y objetivo

El problema abordado es la necesidad de vincular los registros de asistencia con el aprendiz y la sesión correspondientes, facilitar su consulta y reducir inconsistencias operativas. No se dispone de una línea base cuantitativa que permita estimar la magnitud de estos problemas en una población institucional determinada.

El objetivo del proyecto es desarrollar un sistema de gestión de asistencia académica con acceso por rol e integración de reconocimiento facial. Sus objetivos específicos comprenden organizar los datos académicos, autenticar usuarios, integrar el registro y la comparación del rostro, permitir consultas de asistencia y documentar la solución para su mantenimiento.

Usuarios y alcance

Rol	Participación en el sistema
Coordinador	Gestiona usuarios, información y asociaciones académicas; participa en la administración de solicitudes.
Instructor	Trabaja con fichas asignadas, configura el contexto de sesiones y opera o consulta el proceso de asistencia.
Aprendiz	Registra su rostro, consulta el perfil y los registros propios y utiliza solicitudes disponibles.

La carpeta admin del frontend corresponde al coordinador; no representa un cuarto rol. La arquitectura descrita no depende de Raspberry Pi y no presenta el módulo de horarios retirado como una funcionalidad activa. El procesamiento del rostro se realiza mediante un microservicio Python.

2. Metodología de desarrollo

Se describe un proceso de desarrollo organizado por funcionalidades, apoyado en los documentos y repositorios suministrados. No se atribuye una metodología ágil específica, un número de sprints o un protocolo experimental que no estén acreditados en las fuentes del proyecto.

Análisis y modelado

El SRS define actores, requisitos, reglas y condiciones de operación. Las historias de usuario traducen necesidades de los roles en funcionalidades y criterios de aceptación; MoSCoW permite delimitar prioridades de entrega. Los modelos de dominio y de datos identifican conceptos como aprendiz, ficha, sesión y asistencia. Estos artefactos orientan la implementación, pero su existencia no demuestra por sí sola que todos los requisitos estén completos.

Implementación e integración

La construcción se distribuye entre persistencia, servicios de negocio, interfaz y procesamiento facial. Las migraciones preparan el esquema PostgreSQL; Java implementa contratos HTTP y reglas; Expo presenta las operaciones; Python procesa capturas y compara vectores. La integración requiere coherencia entre identificadores, estados, asociaciones y respuestas de cada componente.

Revisión y correcciones

El equipo reportó tres problemas corregidos: dificultades de visualización de la cámara, registros incompletos y errores de traducción. La documentación relaciona las correcciones con ajustes del flujo de captura y registro y con el uso de recursos de internacionalización. Sin resultados de pruebas adjuntos, estas mejoras se presentan como reportadas y no como evidencia de una reducción cuantificada de errores.

Criterios de evaluación

Criterio	Evidencia que permite evaluarlo
Integridad	El evento identifica al aprendiz y la sesión y respeta restricciones de duplicidad.
Seguridad funcional	Cada rol accede únicamente a las operaciones y recursos autorizados.
Usabilidad	La cámara, la carga y los mensajes permiten completar el recorrido sin ambigüedad.
Integración	La confirmación visual corresponde a una respuesta remota y a datos persistidos.
Mantenibilidad	Los contratos, módulos, migraciones y configuración permiten reconstruir el entorno.

La revisión disponible es principalmente documental y estática. La evaluación experimental deberá registrar escenarios, versión, dispositivos, datos de prueba y resultados reproducibles.

3. Desarrollo de la solución

Backend y arquitectura en capas

El backend Java 17 con Spring Boot 3.4.1 se ejecuta como una sola aplicación: es un monolito modular con arquitectura en capas. Los paquetes auth y academic agrupan funcionalidades; config y shared aportan capacidades transversales. Dentro de cada módulo, el controlador recibe HTTP, el servicio coordina reglas y el repositorio accede a datos con JPA. Entidades, DTOs y excepciones apoyan ese recorrido. Las capas son responsabilidades lógicas, no servidores independientes.

Esta organización permite localizar cambios y separar contratos, reglas y persistencia sin desplegar cada función académica como un servicio distinto. Los métodos de fábrica uniforman respuestas; Jwts.builder() construye tokens; los servicios actúan como fachadas de casos de uso. Spring gestiona beans singleton, InstructorTypeConverter adapta valores del dominio a la base y AuditBase comparte campos de auditoría. JwtFilter es un interceptor de seguridad, no un Proxy GoF puro.

Frontend y flujo de interacción

React Native, Expo y TypeScript conforman una interfaz organizada en app/ para pantallas y rutas, features/ para lógica de cada funcionalidad y shared/ para componentes, contextos y servicios reutilizables. Los hooks coordinan formularios y estados; AuthContext comparte la sesión; Axios realiza solicitudes. No todas las acciones atraviesan un contexto: algunas utilizan directamente un servicio del módulo.

En el acceso, la pantalla y el hook validan datos, el servicio solicita autenticación, Java devuelve token y rol y la aplicación abre el panel correspondiente. El cliente añade el JWT a solicitudes protegidas. Ocultar controles no sustituye la autorización del servidor.

Persistencia y configuración

PostgreSQL 16 conserva relaciones académicas y biométricas. Liquibase aplica cambios versionados por separado; Hibernate utiliza ddl-auto: validate. Los esquemas organizan objetos por dominio y las claves relacionan programas, fichas, instructores, aprendices y sesiones. pgvector permite almacenar y comparar vectores en PostgreSQL [3].

application.yaml concentra conexión, correo, JWT e integración facial. Las credenciales se inyectan mediante variables del entorno; las variables EXPO_PUBLIC configuran el cliente y no son un lugar seguro para secretos. El correo contempla recuperación de contraseña y notificaciones con despacho asíncrono documentado por el equipo.

4. Face Mesh y reconocimiento facial

Función de la malla facial

El equipo informa el uso de Face Mesh, de MediaPipe, como biblioteca de apoyo para implementar la etapa de análisis facial. Su función es localizar puntos de referencia y representar la geometría del rostro durante la captura. La documentación de MediaPipe describe la detección de landmarks y la estimación de superficies faciales en imágenes o vídeo [1]. La malla ayuda a describir la posición del rostro; por sí sola no identifica al aprendiz.

Esta incorporación se incluye conforme a la actualización aportada por el equipo. La versión exacta, la plataforma de ejecución y los archivos que la integran no fueron proporcionados en esta actualización; por ello no se atribuyen controles de captura concretos ni una protección contra suplantación demostrada.

Función de InsightFace y del microservicio

La implementación facial anteriormente revisada utiliza FastAPI e InsightFace, con ONNX Runtime, OpenCV y NumPy. InsightFace ofrece herramientas de detección, alineación y reconocimiento facial [2]. El worker genera un embedding: un vector numérico utilizado para comparar la captura con rostros registrados. Esta representación cumple una función diferente de la malla de puntos.

Componente	Función en el flujo
Face Mesh / MediaPipe	Apoyo al análisis geométrico de la captura; uso informado por el equipo.
InsightFace	Generación de embeddings en el microservicio revisado.
pgvector	Almacenamiento y comparación de los vectores faciales.
Backend Java	Permisos, reglas de sesión y coordinación del evento de asistencia.

Enrolamiento y verificación

El enrolamiento vincula el rostro con el aprendiz. El frontend revisado llama directamente al servicio Python para esta operación. En la asistencia por sesión, Java dispone de un cliente de verificación hacia Python. El servicio genera un vector temporal y compara candidatos activos de la ficha correspondiente; el resultado vuelve al flujo de asistencia.

La similitud se evalúa frente a un umbral; el valor por defecto observado, 0,65, no representa un porcentaje de precisión. Registrar el rostro y registrar asistencia son operaciones distintas. La detección de una malla, la comparación de identidad y la prueba de presencia real requieren validaciones diferenciadas.

5. Resultados y discusión

Resultados técnicos documentados

Los artefactos disponibles describen una solución con tres roles, datos académicos relacionados, contratos de autenticación, interfaz compartida y procesamiento facial separado. La documentación del backend V5 aporta patrones localizados, configuración externa y despacho de correo. Estos resultados evidencian diseño y código documentado, sin certificar por sí mismos todos los recorridos en producción.

El esquema incorpora medidas de integridad, entre ellas un índice único parcial para eventos activos por sesión, aprendiz y tipo. El equipo reporta la corrección de problemas de cámara, registro y traducciones. No se dispone aquí de mediciones antes y después, conjuntos de imágenes etiquetados ni resultados experimentales de exactitud, latencia o disponibilidad.

Despliegue y límites de la evidencia

FaceLit está desplegado íntegramente en el VPS Hostinger: Caddy publica https://facelit.tech/ y actúa como proxy de API/WebSocket. Docker Compose ejecuta PostgreSQL 16 con pgvector, Spring Boot y FastAPI en red privada. El equipo confirma acceso, CSV, correo, cámara y asistencia. Actualización: 5 de noviembre de 2026.

Discusión

La separación del procesamiento Python facilita gestionar dependencias del modelo sin incorporarlas al proceso Java. Sin embargo, ambos servicios comparten datos y deben coordinar cambios de esquema. El acceso facial directo desde el cliente también exige revisar autorización y gestión de credenciales: una clave incluida en el frontend puede ser accesible al usuario.

Face Mesh aporta información geométrica y puede apoyar la experiencia de captura, mientras los embeddings permiten comparar identidad. Ninguna de estas funciones acredita automáticamente una defensa contra fotografías o vídeo. La calidad debe evaluarse con diversidad de iluminación, poses, cámaras y usuarios, registrando falsos positivos y negativos. Asimismo, el uso biométrico requiere minimizar información, delimitar acceso y documentar consentimiento y conservación.

6. Conclusiones

FaceLit integra gestión académica, control de acceso y asistencia con procesamiento facial en una arquitectura de componentes diferenciados. El monolito Java organiza las reglas por funcionalidad y capas; Expo separa interfaz, lógica y comunicación; PostgreSQL conserva relaciones y vectores; Python realiza la comparación del rostro. Face Mesh se incorpora, según lo informado por el equipo, como apoyo de malla facial y no como sustituto de InsightFace.

La contribución es una base técnica y documental para organizar el seguimiento de asistencia. Confirmar su eficacia requiere pruebas integrales, mediciones reproducibles y evidencia del despliegue. Los resultados cuantitativos deberán publicarse únicamente cuando existan datos que los sustenten.

Referencias bibliográficas

[1] Google. (s. f.). Face landmark detection guide. MediaPipe, Google AI Edge. Consulta: 9 de octubre de 2026.
developers.google.com/edge/mediapipe/solutions/vision/face_landmarker

[2] DeepInsight. (s. f.). InsightFace: 2D and 3D face analysis project. Repositorio oficial. Consulta: 9 de octubre de 2026.
github.com/deepinsight/insightface

[3] pgvector. (s. f.). Open-source vector similarity search for Postgres. Repositorio oficial. Consulta: 9 de octubre de 2026.
github.com/pgvector/pgvector

[4] Equipo FaceLit. (2026). SRS V3 completo y trazable. Documento interno del proyecto, suministrado por el equipo.

[5] Equipo FaceLit. (2026). FaceLit-DB, FaceLit-Backend, FaceLit y micro-facial. Repositorios y archivos de implementación suministrados para la revisión del proyecto.

[6] Equipo FaceLit. (2026). Implementación y despliegue en Hostinger. Documento interno del proyecto, carpeta 17_Despliegue; actualización del 4 de noviembre de 2026.

[7] Equipo FaceLit. (2026). Patrones de diseño y calidad de código; documentación del frontend; seguridad, correo y configuración del backend V5. Material técnico aportado por el equipo.

[8] Equipo FaceLit. (2026). Actualización sobre el uso de Face Mesh. Información comunicada por el equipo para la elaboración de este artículo.

Nota sobre las fuentes del proyecto

Las fuentes internas [4]-[8] describen el proyecto y su evolución; no constituyen publicaciones científicas externas. La atribución de Face Mesh procede de la actualización del equipo y se distingue de la implementación del microservicio previamente revisada. El artículo conserva esta diferencia para evitar confundir una incorporación informada con una integración nuevamente verificada.